



MODUL PRAKTIKUM

Disusun Oleh:

Aprilia Kurnianti, S.T., M.Eng.

Dosen Pengampuh Matkul IBD

PRAKTIKUM 4

Subquery, Common Table Expression (CTE), dan Window Function

Tujuan Praktikum

Setelah menyelesaikan praktikum ini mahasiswa mampu:

1. Membuat query di dalam query menggunakan Subquery.
2. Membentuk Result Set sementara menggunakan CTE.
3. Memberikan nomor urut dan peringkat menggunakan Window Function.
4. Membagi data menjadi beberapa kelompok menggunakan NTILE.
5. Membandingkan data saat ini dengan data sebelumnya menggunakan LAG.
6. Mengakses data berikutnya menggunakan LEAD.

Persiapan

- Aktifkan database.

Catatan Kecukupan Data

Fungsi seperti RANK, DENSE_RANK, NTILE, LAG, dan LEAD lebih mudah diamati apabila tabel memiliki cukup banyak data.

Menambahkan data minimal:

10 pelanggan;

10 produk;

5 pegawai;

5 layanan;

15 transaksi penjualan;

25 detail penjualan;

15 reservasi layanan;

Beberapa catatan audit untuk produk yang sama.

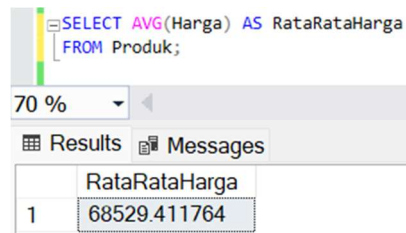
Jika data hanya terdiri atas tiga pelanggan atau satu transaksi, beberapa query tetap dapat dijalankan, tetapi perbedaan hasil antarfungsi belum terlihat jelas.

Praktikum 1 - Scalar Subquery

1. Menghitung Rata-Rata Harga

Jalankan query berikut terlebih dahulu:

```
SELECT AVG(Harga) AS RataRataHarga
FROM Produk;
```



The screenshot shows a SQL query interface with a search bar at 70% zoom. Below the query editor, there are tabs for 'Results' and 'Messages'. The 'Results' tab is active, displaying a single row of data. The column is named 'RataRataHarga' and the value is '68529.411764'.

	RataRataHarga
1	68529.411764

Query tersebut menghasilkan satu nilai sehingga termasuk Scalar Query.

2. Produk dengan Harga di Atas Rata-Rata

```
SELECT
    KodeProduk,
    NamaProduk,
    Harga,
    Stok
FROM Produk
WHERE Harga >
(
    SELECT AVG(Harga)
    FROM Produk
)
ORDER BY Harga DESC;
```

3. Menampilkan Rata-Rata sebagai Kolom Tambahan

```
SELECT
    KodeProduk,
    NamaProduk,
    Harga,
    (
        SELECT AVG(Harga)
        FROM Produk
    ) AS RataRataHarga
FROM Produk
ORDER BY Harga DESC;
```

4. Menampilkan Selisih Harga

```
SELECT
    KodeProduk,
    NamaProduk,
    Harga,
    (
        SELECT AVG(Harga)
        FROM Produk
    ) AS RataRataHarga,
    Harga -
    (
        SELECT AVG(Harga)
        FROM Produk
    ) AS SelisihHarga
FROM Produk
ORDER BY SelisihHarga DESC;
```

Praktikum 2 - Multiple-Row Subquery

1. Daftar Kode Pelanggan dalam Penjualan

```
SELECT DISTINCT KodePelanggan  
FROM Penjualan;
```

Query tersebut dapat menghasilkan lebih dari satu baris.

2. Pelanggan yang Pernah Bertransaksi

```
SELECT  
    KodePelanggan,  
    NamaPelanggan,  
    Alamat,  
    NoHP  
FROM Pelanggan  
WHERE KodePelanggan IN  
(  
    SELECT KodePelanggan  
    FROM Penjualan  
)  
ORDER BY NamaPelanggan;
```

3. Pelanggan yang Belum Pernah Bertransaksi

```
SELECT  
    KodePelanggan,  
    NamaPelanggan,  
    Alamat,  
    NoHP  
FROM Pelanggan  
WHERE KodePelanggan NOT IN  
(  
    SELECT KodePelanggan  
    FROM Penjualan  
)  
ORDER BY NamaPelanggan;
```

Karena Penjualan.KodePelanggan didefinisikan sebagai NOT NULL. Penggunaan NOT IN pada kasus ini relatif aman dari masalah nilai NULL.

4. Produk yang Pernah Terjual

```
SELECT  
    KodeProduk,  
    NamaProduk,  
    Harga,  
    Stok  
FROM Produk  
WHERE KodeProduk IN  
(  
    SELECT KodeProduk  
    FROM DetailPenjualan  
)  
ORDER BY NamaProduk;
```

5. Produk yang Belum Pernah Terjual

```
SELECT  
    KodeProduk,  
    NamaProduk,  
    Harga,  
    Stok  
FROM Produk  
WHERE KodeProduk NOT IN  
(  
    SELECT KodeProduk  
    FROM DetailPenjualan  
)  
ORDER BY NamaProduk;
```

6. Pelanggan yang Pernah Membeli dan Melakukan Reservasi

```
SELECT
    KodePelanggan,
    NamaPelanggan
FROM Pelanggan
WHERE KodePelanggan IN
(
    SELECT KodePelanggan
    FROM Penjualan
)
AND KodePelanggan IN
(
    SELECT KodePelanggan
    FROM ReservasiLayanan
);
```

Praktikum 3 - EXISTS dan NOT EXISTS

Tujuannya agar mahasiswa mampu memeriksa keberadaan hubungan data menggunakan subquery.

1. Pelanggan yang Memiliki Hewan

```
SELECT
    p.KodePelanggan,
    p.NamaPelanggan
FROM Pelanggan AS p
WHERE EXISTS
(
    SELECT 1
    FROM Hewan AS h
    WHERE h.KodePelanggan = p.KodePelanggan
);
```

2. Pelanggan yang Tidak Memiliki Hewan

```
SELECT
    p.KodePelanggan,
    p.NamaPelanggan
FROM Pelanggan AS p
WHERE NOT EXISTS
(
    SELECT 1
    FROM Hewan AS h
    WHERE h.KodePelanggan = p.KodePelanggan
);
```

3. Produk yang Pernah Terjual

```
SELECT
    pr.KodeProduk,
    pr.NamaProduk,
    pr.Harga
FROM Produk AS pr
WHERE EXISTS
(
    SELECT 1
    FROM DetailPenjualan AS dp
    WHERE dp.KodeProduk = pr.KodeProduk
);
```

Pada EXISTS, SQL Server hanya memeriksa apakah terdapat minimal satu baris yang memenuhi kondisi. Nilai yang ditulis setelah SELECT pada subquery tidak menjadi fokus utama. Oleh sebab itu, SELECT 1 sering digunakan untuk menunjukkan bahwa yang diperiksa hanyalah keberadaan data.

Praktikum 4 - Correlated Subquery

1. Menghitung rata-rata yang berbeda untuk setiap kategori produk.

```
SELECT
    pr.KodeProduk,
    pr>NamaProduk,
    pr.KodeKategori,
    pr.Harga
FROM Produk AS pr
WHERE pr.Harga >
(
    SELECT AVG(pr2.Harga)
    FROM Produk AS pr2
    WHERE pr2.KodeKategori = pr.KodeKategori
)
ORDER BY pr.KodeKategori, pr.Harga DESC;
```

2. Menampilkan Rata-Rata Kategori

```
SELECT
    pr.KodeProduk,
    pr>NamaProduk,
    pr.KodeKategori,
    pr.Harga,
    (
        SELECT AVG(pr2.Harga)
        FROM Produk AS pr2
        WHERE pr2.KodeKategori = pr.KodeKategori
    ) AS RataRataHargaKategori
FROM Produk AS pr
ORDER BY pr.KodeKategori, pr.Harga DESC;
```

3. Pelanggan dengan Lebih dari Satu Hewan

```
SELECT
    p.KodePelanggan,
    p>NamaPelanggan
FROM Pelanggan AS p
WHERE
(
    SELECT COUNT(*)
    FROM Hewan AS h
    WHERE h.KodePelanggan = p.KodePelanggan
) > 1;
```

Praktikum 5 - CTE Dasar

1. Daftar Produk dan Kategori

```
WITH DataProduk AS
(
    SELECT
        pr.KodeProduk,
        pr>NamaProduk,
        kp>NamaKategori,
        pr.Harga,
        pr.Stok
    FROM Produk AS pr
    INNER JOIN KategoriProduk AS kp
        ON pr.KodeKategori = kp.KodeKategori
)
SELECT
    KodeProduk,
   >NamaProduk,
   >NamaKategori,
   >Harga,
   >Stok
FROM DataProduk
ORDER BY>NamaKategori,>NamaProduk;
```

2. Produk dengan Harga di Atas Rata-Rata

```
WITH RataRataProduk AS
(
    SELECT AVG(Harga) AS RataRataHarga
    FROM Produk
)
SELECT
    pr.KodeProduk,
    pr>NamaProduk,
    pr.Harga,
    rr.RataRataHarga,
    pr.Harga - rr.RataRataHarga AS SelisihHarga
FROM Produk AS pr
CROSS JOIN RataRataProduk AS rr
WHERE pr.Harga > rr.RataRataHarga
ORDER BY pr.Harga DESC;
```

Praktikum 6 - CTE dengan Agregasi

1. Total Setiap Transaksi

```
WITH TotalTransaksi AS
(
    SELECT
        dp.KodePenjualan,
        SUM((dp.Jumlah * dp.HargaJual) - dp.Diskon) AS TotalPenjualan
    FROM DetailPenjualan AS dp
    GROUP BY dp.KodePenjualan
)
SELECT
    pj.KodePenjualan,
    pj.TanggalPenjualan,
    p>NamaPelanggan,
    pg>NamaPegawai,
    tt.TotalPenjualan,
    pj.MetodeBayar
FROM TotalTransaksi AS tt
INNER JOIN Penjualan AS pj
    ON tt.KodePenjualan = pj.KodePenjualan
INNER JOIN Pelanggan AS p
    ON pj.KodePelanggan = p.KodePelanggan
INNER JOIN Pegawai AS pg
    ON pj.KodePegawai = pg.KodePegawai
ORDER BY tt.TotalPenjualan DESC;
```

2. Total Pembelian Setiap Pelanggan

```
WITH TotalTransaksi AS
(
    SELECT
        dp.KodePenjualan,
        SUM((dp.Jumlah * dp.HargaJual) - dp.Diskon) AS TotalPenjualan
    FROM DetailPenjualan AS dp
    GROUP BY dp.KodePenjualan
),
TotalPelanggan AS
(
    SELECT
        pj.KodePelanggan,
        COUNT(pj.KodePenjualan) AS JumlahTransaksi,
        SUM(tt.TotalPenjualan) AS TotalPembelian
    FROM Penjualan AS pj
    INNER JOIN TotalTransaksi AS tt
        ON pj.KodePenjualan = tt.KodePenjualan
    GROUP BY pj.KodePelanggan
)
SELECT
    p.KodePelanggan,
    p>NamaPelanggan,
    COALESCE(tp.JumlahTransaksi, 0) AS JumlahTransaksi,
    COALESCE(tp.TotalPembelian, 0) AS TotalPembelian
FROM Pelanggan AS p
LEFT JOIN TotalPelanggan AS tp
    ON p.KodePelanggan = tp.KodePelanggan
ORDER BY TotalPembelian DESC;
```

Praktikum 7 - CTE untuk Audit Data

1. Produk yang Belum Pernah Terjual

```
WITH ProdukTerjual AS
(
    SELECT DISTINCT KodeProduk
    FROM DetailPenjualan
)
SELECT
    pr.KodeProduk,
    pr>NamaProduk,
    pr.Harga,
    pr.Stok
FROM Produk AS pr
LEFT JOIN ProdukTerjual AS pt
    ON pr.KodeProduk = pt.KodeProduk
WHERE pt.KodeProduk IS NULL;
```

2. Riwayat Audit Produk

```
WITH RiwayatAudit AS
(
    SELECT
        a.KodeAudit,
        a.KodeProduk,
        a.TanggalAudit,
        a.StokSebelum,
        a.StokSesudah,
        a.StokSesudah - a.StokSebelum AS PerubahanStok,
        a.Keterangan
    FROM AuditStok AS a
)
SELECT
    ra.KodeAudit,
    pr>NamaProduk,
    ra.TanggalAudit,
    ra.StokSebelum,
    ra.StokSesudah,
    ra.PerubahanStok,
    ra.Keterangan
FROM RiwayatAudit AS ra
INNER JOIN Produk AS pr
    ON ra.KodeProduk = pr.KodeProduk
ORDER BY ra.TanggalAudit DESC;
```

Praktikum 8 - Fungsi Agregasi dengan OVER

1. Total Stok Seluruh Produk

```
SELECT
    KodeProduk,
    NamaProduk,
    Stok,
    SUM(Stok) OVER () AS TotalSeluruhStok
FROM Produk
ORDER BY KodeProduk;
```

Setiap produk tetap ditampilkan, sedangkan total stok seluruh produk ditambahkan pada setiap baris.

2. Agregasi Berdasarkan Kategori

```
SELECT
    pr.KodeProduk,
    pr.NamaProduk,
    kp.NamaKategori,
    pr.Harga,
    AVG(pr.Harga) OVER
    (
        PARTITION BY pr.KodeKategori
    ) AS RataRataHargaKategori,
    SUM(pr.Stok) OVER
    (
        PARTITION BY pr.KodeKategori
    ) AS TotalStokKategori
FROM Produk AS pr
INNER JOIN KategoriProduk AS kp
    ON pr.KodeKategori = kp.KodeKategori
ORDER BY kp.NamaKategori, pr.Harga DESC;
```

3. Perbandingan GROUP BY dan OVER

- Menggunakan GROUP BY

```
SELECT
    KodeKategori,
    AVG(Harga) AS RataRataHarga
FROM Produk
GROUP BY KodeKategori;
```

70 %

Results Messages

	KodeKategori	RataRataHarga
1	KTG001	97500.000000
2	KTG002	27500.000000
3	KTG003	45000.000000
4	KTG101	75000.000000
5	KTG102	95000.000000
6	KTG103	45000.000000
7	KTG104	75000.000000

- Menggunakan OVER

```

SELECT
    KodeProduk,
    NamaProduk,
    KodeKategori,
    Harga,
    AVG(Harga) OVER
    (
        PARTITION BY KodeKategori
    ) AS RataRataHargaKategori
FROM Produk;

```

	KodeProduk	NamaProduk	KodeKategori	Harga	RataRataHargaKategori
1	PRD001	Whiskas Tuna 1kg	KTG001	75000.00	97500.000000
2	PRD002	Dog Food Premium 2kg	KTG001	120000.00	97500.000000
3	PRD003	Kalung Kucing	KTG002	25000.00	27500.000000
4	PRD005	Mainan Bola Kucing	KTG002	30000.00	27500.000000
5	PRD004	Vitamin Bulu Kucing	KTG003	45000.00	45000.000000
6	PRD101	Cat Food Tuna 1 Kg	KTG101	85000.00	75000.000000
7	PRD102	Cat Food Salmon 1 Kg	KTG101	95000.00	75000.000000
8	PRD103	Cat Food Chicken 1 Kg	KTG101	85000.00	75000.000000
9	PRD104	Wet Food Tuna 400 Gr	KTG101	35000.00	75000.000000
10	PRD105	Dog Food Beef 1 Kg	KTG102	90000.00	95000.000000
11	PRD106	Dog Food Chicken 1 Kg	KTG102	90000.00	95000.000000
12	PRD107	Dog Food Puppy 1 Kg	KTG102	105000.00	95000.000000

GROUP BY mengurangi jumlah baris, sedangkan OVER mempertahankan detail untuk setiap produk.

Praktikum 9 - ROW_NUMBER

1. Nomor Urut Produk Berdasarkan Harga

```

SELECT
    ROW_NUMBER() OVER
    (
        ORDER BY Harga DESC, KodeProduk ASC
    ) AS NomorUrut,
    KodeProduk,
    NamaProduk,
    Harga,
    Stok
FROM Produk;

```

	NomorUrut	KodeProduk	NamaProduk	Harga	Stok
1	1	PRD002	Dog Food Premium 2kg	120000.00	20
2	2	PRD107	Dog Food Puppy 1 Kg	105000.00	25
3	3	PRD102	Cat Food Salmon 1 Kg	95000.00	32
4	4	PRD105	Dog Food Beef 1 Kg	90000.00	30
5	5	PRD106	Dog Food Chicken 1 Kg	90000.00	35
6	6	PRD101	Cat Food Tuna 1 Kg	85000.00	40
7	7	PRD103	Cat Food Chicken 1 Kg	85000.00	45
8	8	PRD001	Whiskas Tuna 1kg	75000.00	30
9	9	PRD111	Vitamin Bulu Hewan	75000.00	24

KodeProduk ditambahkan pada pengurutan agar hasil lebih konsisten ketika terdapat harga yang sama.

2. Reservasi Terbaru Setiap Pelanggan

```
WITH UrutanReservasi AS
(
    SELECT
        r1.KodeReservasi,
        r1.KodePelanggan,
        r1.TanggalReservasi,
        r1.JamReservasi,
        r1.KodeLayanan,
        r1.StatusReservasi,
        ROW_NUMBER() OVER
        (
            PARTITION BY r1.KodePelanggan
            ORDER BY
                r1.TanggalReservasi DESC,
                r1.JamReservasi DESC,
                r1.KodeReservasi DESC
        ) AS NomorUrut
    FROM ReservasiLayanan AS r1
)
SELECT
    ur.KodeReservasi,
    p>NamaPelanggan,
    ur.TanggalReservasi,
    ur.JamReservasi,
    l>NamaLayanan,
    ur.StatusReservasi
FROM UrutanReservasi AS ur
INNER JOIN Pelanggan AS p
    ON ur.KodePelanggan = p.KodePelanggan
INNER JOIN Layanan AS l
    ON ur.KodeLayanan = l.KodeLayanan
WHERE ur.NomorUrut = 1;
```

Praktikum 10 - RANK dan DENSE_RANK

1. Perbandingan Peringkat Harga Produk

```
SELECT
    KodeProduk,
    NamaProduk,
    Harga,
    ROW_NUMBER() OVER
    (
        ORDER BY Harga DESC, KodeProduk
    ) AS NomorUrut,
    RANK() OVER
    (
        ORDER BY Harga DESC
    ) AS PeringkatRank,
    DENSE_RANK() OVER
    (
        ORDER BY Harga DESC
    ) AS PeringkatDenseRank
FROM Produk
ORDER BY Harga DESC, KodeProduk;
```

	KodeProduk	NamaProduk	Harga	NomorUrut	PeringkatRank	PeringkatDenseRank
1	PRD002	Dog Food Premium 2kg	120000.00	1	1	1
2	PRD107	Dog Food Puppy 1 Kg	105000.00	2	2	2
3	PRD102	Cat Food Salmon 1 Kg	95000.00	3	3	3
4	PRD105	Dog Food Beef 1 Kg	90000.00	4	4	4
5	PRD106	Dog Food Chicken 1 Kg	90000.00	5	4	4
6	PRD101	Cat Food Tuna 1 Kg	85000.00	6	6	5
7	PRD103	Cat Food Chicken 1 Kg	85000.00	7	6	5
8	PRD001	Whiskas Tuna 1kg	75000.00	8	8	6
9	PRD111	Vitamin Bulu Hewan	75000.00	9	8	6
10	PRD112	Sampo Antikutu	75000.00	10	8	6

Perbedaan akan terlihat apabila terdapat 2 produk atau lebih dengan harga yang sama.

2. Peringkat Produk Berdasarkan Jumlah Terjual

```
WITH PenjualanProduk AS
(
    SELECT
        pr.KodeProduk,
        pr>NamaProduk,
        COALESCE(SUM(dp.Jumlah), 0) AS TotalTerjual
    FROM Produk AS pr
    LEFT JOIN DetailPenjualan AS dp
        ON pr.KodeProduk = dp.KodeProduk
    GROUP BY
        pr.KodeProduk,
        pr>NamaProduk
)
SELECT
    KodeProduk,
   >NamaProduk,
   >TotalTerjual,
    RANK() OVER
    (
        ORDER BY TotalTerjual DESC
    ) AS PeringkatRank,
    DENSE_RANK() OVER
    (
        ORDER BY TotalTerjual DESC
    ) AS PeringkatDenseRank
FROM PenjualanProduk
ORDER BY TotalTerjual DESC,>NamaProduk;
```

70 %

Results Messages

	KodeProduk	NamaProduk	TotalTerjual	PeringkatRank	PeringkatDenseRank
1	PRD004	Vitamin Bulu Kucing	2	1	1
2	PRD001	Whiskas Tuna 1kg	2	1	1
3	PRD002	Dog Food Premium 2kg	1	3	2
4	PRD003	Kalung Kucing	1	3	2
5	PRD005	Mainan Bola Kucing	1	3	2

Praktikum 11 – NTILE

Membagi Produk Menjadi Empat Kelompok Harga

```
SELECT
    KodeProduk,
   >NamaProduk,
   >Harga,
    NTILE(4) OVER
    (
        ORDER BY Harga DESC, KodeProduk
    ) AS KelompokHarga
FROM Produk
ORDER BY KelompokHarga, Harga DESC;
```

70 %

Results Messages

	KodeProduk	NamaProduk	Harga	KelompokHarga
1	PRD002	Dog Food Premium 2kg	120000.00	1
2	PRD107	Dog Food Puppy 1 Kg	105000.00	1
3	PRD102	Cat Food Salmon 1 Kg	95000.00	1
4	PRD105	Dog Food Beef 1 Kg	90000.00	1
5	PRD106	Dog Food Chicken 1 Kg	90000.00	1
6	PRD101	Cat Food Tuna 1 Kg	85000.00	2
7	PRD103	Cat Food Chicken 1 Kg	85000.00	2
8	PRD001	Whiskas Tuna 1kg	75000.00	2
9	PRD111	Vitamin Bulu Hewan	75000.00	2
10	PRD112	Sampo Antikutu	75000.00	3
11	PRD110	Tempat Makan Stainl...	65000.00	3
12	PRD004	Vitamin Bulu Kucing	45000.00	3
13	PRD108	Kalung Hewan Premium	45000.00	3
14	PRD104	Wet Food Tuna 400 Gr	35000.00	4
15	PRD005	Mainan Bola Kucing	30000.00	4
16	PRD003	Kalung Kucing	25000.00	4
17	PRD109	Bola Mainan Hewan	25000.00	4

Kelompok satu berisi produk dengan posisi harga tertinggi karena pengurutan menggunakan DESC.

Praktikum 12 – LAG

Membandingkan Harga Produk dengan Harga Kode Produk Sebelumnya

```
SELECT
    KodeProduk,
    NamaProduk,
    Harga,
    LAG(Harga, 1) OVER
    (
        ORDER BY Harga ASC, KodeProduk
    ) AS HargaSebelumnya,
    Harga - LAG(Harga, 1) OVER
    (
        ORDER BY Harga ASC, KodeProduk
    ) AS SelisihHarga
FROM Produk
ORDER BY Harga ASC, KodeProduk;
```

70 %

Results Messages

	KodeProduk	NamaProduk	Harga	HargaSebelumnya	SelisihHarga
1	PRD003	Kalung Kucing	25000.00	NULL	NULL
2	PRD109	Bola Mainan Hewan	25000.00	25000.00	0.00
3	PRD005	Mainan Bola Kucing	30000.00	25000.00	5000.00
4	PRD104	Wet Food Tuna 400 Gr	35000.00	30000.00	5000.00
5	PRD004	Vitamin Bulu Kucing	45000.00	35000.00	10000.00
6	PRD108	Kalung Hewan Premium	45000.00	45000.00	0.00

Praktikum 13 – LEAD

Reservasi Berikutnya Setiap Pegawai. diTentukan Berdasarkan Jadwal Kode Reservasi Setelahnya.

```
SELECT
    r1.KodePegawai,
    pg.NamaPegawai,
    r1.KodeReservasi,
    r1.TanggalReservasi,
    r1.JamReservasi,
    LEAD(r1.TanggalReservasi, 1) OVER
    (
        PARTITION BY r1.KodePegawai
        ORDER BY
            r1.TanggalReservasi,
            r1.JamReservasi,
            r1.KodeReservasi
    ) AS TanggalReservasiBerikutnya,
    LEAD(r1.JamReservasi, 1) OVER
    (
        PARTITION BY r1.KodePegawai
        ORDER BY
            r1.TanggalReservasi,
            r1.JamReservasi,
            r1.KodeReservasi
    ) AS JamReservasiBerikutnya
FROM ReservasiLayanan AS r1
INNER JOIN Pegawai AS pg
    ON r1.KodePegawai = pg.KodePegawai
ORDER BY
    r1.KodePegawai,
    r1.TanggalReservasi,
    r1.JamReservasi;
```